Cloud Computing Comprehensive Revision Notes

18 Topics — Explained Simply with Examples

Part 1 — Core Cloud Concepts

Utility Computing · Containers vs VMs · SOAP in XML · SOAP + HTTP
Network Virtualization · Onsite/Outside Private & Community Cloud

Part 2 — Platforms & Data

OpenStack · Amazon Dynamo · Relational vs NoSQL Databases
Microsoft Azure · Google Cloud Platform (GCP)

Part 3 — Cloud Management & Optimization

Physical & Logical Resources · Energy Management · VM Scheduling
VM Management · Overhead in Cloud Computing

Part 4 — Cloud Security

Security Issues · 4 Types of Security Attacks
Multi-Tenancy · Threat Model · Attack Model · Network Probing ·
Co-Residency

April 24, 2026 • Beginner-Friendly with Analogies, Tables & Exam Definitions

Contents

1 Part 1 — Core Cloud Concepts **2**
1.1 Utility Computing . . . 2
1.1.1 Key Features . . . 2
1.1.2 Real-World Examples . . . 2
1.2 Containers vs Virtual Machines — Docker vs Oracle VM VirtualBox . . . 3
1.2.1 Virtual Machines — Oracle VirtualBox . . . 3
1.2.2 Containers — Docker . . . 3
1.2.3 Full Comparison Table . . . 4
1.3 SOAP in XML — Structure and Message Format . . . 5
1.3.1 SOAP Message Structure . . . 5
1.3.2 SOAP Request Example . . . 5
1.3.3 SOAP Response Example . . . 5
1.3.4 Key Points about SOAP . . . 6
1.4 SOAP + HTTP Working Together — Real Website Example . . . 6
1.4.1 Step-by-Step: Banking Website Example . . . 6
1.5 Network Virtualization . . . 8
1.5.1 Real Example: University Campus Network . . . 8
1.5.2 Another Example: Cloud Platforms (AWS VPC) . . . 8
1.5.3 How Network Virtualization Works . . . 8
1.6 Onsite/Outside Private and Community Cloud . . . 9
1.6.1 1. Onsite Private Cloud . . . 9
1.6.2 2. Onsite Community Cloud . . . 9
1.6.3 3. Outside Private Cloud . . . 9
1.6.4 4. Outside Community Cloud . . . 9
1.6.5 Quick Comparison Table . . . 10

2 Part 2 — Platforms & Data **11**
2.1 OpenStack in Cloud Computing . . . 11
2.1.1 Main Components of OpenStack . . . 11
2.1.2 How OpenStack Works — Step by Step . . . 11
2.2 Amazon Dynamo in Cloud Computing . . . 12
2.2.1 Key Concepts — Simply Explained . . . 12
2.3 Relational vs Non-Relational Databases . . . 12
2.3.1 Relational Database (SQL) . . . 13
2.3.2 Non-Relational Database (NoSQL) . . . 13
2.3.3 Comparison Table . . . 14
2.4 Microsoft Azure . . . 15
2.4.1 Key Azure Services by Category . . . 15
2.4.2 Real-World Example . . . 15
2.5 Google Cloud Platform (GCP) Services . . . 16

3 Part 3 — Cloud Management & Optimisation **17**
3.1 Types of Resources in Cloud: Physical and Logical . . . 17
3.1.1 Physical Resources — The Real Hardware . . . 17
3.1.2 Logical Resources — The Virtual Layer . . . 17
3.2 Energy Management in Cloud Computing . . . 18

3.2.1 The Problem . . . 18
3.2.2 Energy Management Techniques . . . 18
3.3 VM Scheduling . . . 18
3.3.1 Goals of VM Scheduling . . . 19
3.3.2 Types of Scheduling . . . 19
3.4 VM Management . . . 19
3.4.1 VM Lifecycle Stages . . . 19
3.4.2 VM Migration — A Critical Feature . . . 20
3.5 Overhead in Cloud Computing . . . 20
3.5.1 Sources of Overhead in Cloud . . . 20
3.5.2 How to Reduce Overhead . . . 21

4 Part 4 — Cloud Security 22
4.1 Security Issues in Cloud Computing . . . 22
4.1.1 Major Security Issues . . . 22
4.1.2 How to Mitigate Security Risks . . . 22
4.2 4 Types of Security Attacks . . . 23
4.2.1 1. Interruption — Attack on Availability . . . 23
4.2.2 2. Interception — Attack on Confidentiality . . . 23
4.2.3 3. Modification — Attack on Integrity . . . 23
4.2.4 4. Fabrication — Attack on Authenticity . . . 23
4.2.5 Summary Table . . . 23
4.3 Multi-Tenancy, Threat Model, and Attack Model . . . 25
4.3.1 Multi-Tenancy . . . 25
4.3.2 Threat Model . . . 25
4.3.3 Attack Model . . . 26
4.3.4 How the Three Connect . . . 26
4.4 Network Probing and Co-Residency . . . 26
4.4.1 Network Probing . . . 26
4.4.2 Co-Residency (Co-Location Attack) . . . 27
4.4.3 Network Probing vs Co-Residency Together . . . 28

5 Master Revision Summary — All 18 Topics 30

Part 1 — Core Cloud Concepts

1.1 Utility Computing

What is Utility Computing?

Utility computing is a model where computing resources — processing power, storage, software — are provided to users **on demand**, just like electricity or water. You pay only for what you use, without owning any infrastructure.

Real-Life Analogy

Think of electricity:
- You do not build a power plant at home
- You just switch on the light and pay your monthly bill
- Utility computing works exactly the same way for computing resources

▶ Key Features

- **On-demand access** — resources available whenever needed
- **Pay-as-you-go** — charged based on actual usage
- **Scalability** — easily increase or decrease resources
- **Managed infrastructure** — provider handles maintenance, you just use

▶ Real-World Examples

Utility computing is the foundation of modern cloud platforms:

- Amazon Web Services (AWS) — pay per second for server time
- Microsoft Azure — pay per GB of storage used
- Google Cloud Platform — pay per API call made

Aspect	Traditional IT	Utility Computing
Ownership	Buy and own hardware	Rent on demand
Upfront cost	Very high (CapEx)	Near zero
Scaling	Buy more hardware	Click to add resources
Maintenance	Your responsibility	Provider's responsibility

Key Tip / Memory Trick

One Line: Utility computing = computing as a service, like electricity. Use it, pay for it, don't own it.

1.2 Containers vs Virtual Machines — Docker vs Oracle VM VirtualBox

The Core Question

When you want to run different environments or operating systems on one physical machine, you have two approaches: **Virtual Machines** (full OS inside your computer) or **Containers** (lightweight isolated environments sharing the same OS).

▶ Virtual Machines — Oracle VirtualBox

A VM creates a **complete virtual computer** with its own OS, virtual CPU, virtual RAM, and virtual storage. The VM doesn't know it's virtual — it thinks it's running on real hardware.

Real Example: You + VirtualBox + Kali Linux

- Your real OS = Windows (Host OS)
- Install Oracle VirtualBox (the Hypervisor)
- Inside VirtualBox, run Ubuntu or Kali Linux (Guest OS)
- Result: You now have two complete, separate operating systems running simultaneously on one laptop
- Kali Linux thinks it is on a real physical machine

Advantages of VMs:

- Run completely different OS types (Windows inside Linux, vice versa)
- Strong isolation — like completely separate computers
- Great for OS-level work, security testing, learning

Disadvantages:

- Heavy — each VM needs its own OS (GBs of storage, significant RAM)
- Slow to start — booting a full OS takes minutes

▶ Containers — Docker

Containers do **NOT** run a full OS. Instead, they share the host OS kernel and only package the application plus its dependencies. They are much lighter and faster.

Real Example: Docker for a Web Application

On a single Linux machine with Docker installed, you run:

- Container 1: Node.js backend application
- Container 2: MySQL database
- Container 3: Nginx web server (frontend)

All three run separately and independently — but they all share the same underlying Linux OS kernel. No full OS duplication.

Advantages of Containers:

- Very fast (starts in seconds)
- Lightweight (MBs instead of GBs)
- Perfect for app development and deployment

- Consistent environments across different machines

Disadvantages:

- Cannot easily run completely different OS kernels
- Slightly weaker isolation than VMs

▶ Full Comparison Table

Feature	VM (VirtualBox)	Container (Docker)
OS	Full OS inside	Shares host OS kernel
Size	GBs (heavy)	MBs (light)
Startup time	Minutes (boots OS)	Seconds
Isolation	Strong	Moderate
Use case	Run different OS	Run apps efficiently
Example	Kali Linux on Windows	Node + MySQL + Nginx

Real-Life Analogy

- **Virtual Machine** = Renting a completely separate house — independent, private, but expensive and heavy
- **Container** = Living in separate rooms in the same house — efficient, lighter, but sharing common infrastructure

Key Tip / Memory Trick

VM = OS-level virtualization (whole operating system inside)
Container = Application-level virtualization (just the app and dependencies)

1.3 SOAP in XML — Structure and Message Format

What is SOAP?

SOAP (Simple Object Access Protocol) is a messaging protocol used for exchanging structured information between systems in web services. It uses **XML format** to define the message structure and can be transported over HTTP, SMTP, or other protocols.

▶ SOAP Message Structure

A SOAP message is an XML document with three main parts:

- **Envelope** — the root element, defines the document as SOAP
- **Header** (optional) — metadata such as authentication, routing information
- **Body** — the actual request or response data

▶ SOAP Request Example

```
<?xml version="1.0"?>
<soap:Envelope
    xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">

    <soap:Header>
        <auth>
            <username>user123</username>
            <password>mypass</password>
        </auth>
    </soap:Header>

    <soap:Body>
        <GetUserDetails xmlns="http://example.com/user">
            <UserId>123</UserId>
        </GetUserDetails>
    </soap:Body>
</soap:Envelope>
```

▶ SOAP Response Example

```
<?xml version="1.0"?>
<soap:Envelope
    xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">

    <soap:Body>
        <GetUserDetailsResponse xmlns="http://example.com/user">
            <User>
                <Id>123</Id>
                <Name>Mausam</Name>
                <Email>mausam@example.com</Email>
            </User>
        </GetUserDetailsResponse>
    </soap:Body>
</soap:Envelope>
```

▶ Key Points about SOAP

- SOAP messages are **strictly XML-based** — no other format allowed
- Uses **namespaces** to avoid element name conflicts
- Can travel over HTTP, SMTP, or other transport protocols
- Common in **enterprise systems** and legacy web services
- Provides built-in error handling via the "Fault" element

1.4 SOAP + HTTP Working Together — Real Website Example

The Core Relationship

SOAP and **HTTP** are not competitors — they work together. SOAP defines *what the message looks like*. HTTP defines *how the message travels*.

▶ Step-by-Step: Banking Website Example

Checking Your Bank Balance Online

You log into a banking website and click "Check Balance."
Step 1 — User Action: You click the button. The browser sends a request.
Step 2 — SOAP Creates the XML Message:

```
<soap:Envelope>
  <soap:Body>
    <GetBalance>
      <AccountNumber>12345</AccountNumber>
    </GetBalance>
  </soap:Body>
</soap:Envelope>
```

Step 3 — HTTP Sends the SOAP Message:

```
POST /bankService HTTP/1.1
Host: bank.com
Content-Type: text/xml

<soap:Envelope>...</soap:Envelope>
```

Step 4 — Server Processes: Bank server reads XML, fetches balance from database.
Step 5 — SOAP Response:

```
<soap:Envelope>
  <soap:Body>
    <GetBalanceResponse>
      <Balance>5000.00</Balance>
    </GetBalanceResponse>
  </soap:Body>
</soap:Envelope>
```

Step 6 — HTTP Sends Response Back: Browser receives it and displays "Your Balance: Rs. 5000.00."

Real-Life Analogy

- **SOAP** = The formal letter format (rules for writing the message)
- **HTTP** = The postman delivering the letter (transport service)

Without HTTP, SOAP has no way to travel. Without SOAP, HTTP sends raw unstructured data.

Feature	SOAP	HTTP
Type	Message format protocol	Communication protocol
Role	Defines XML structure	Transfers data over network
Format	Strict XML only	Any format (HTML, JSON, XML)
Use	Web services, enterprise APIs	Web browsing, all web traffic

1.5 Network Virtualization

What is Network Virtualization?

Network virtualization means creating multiple **virtual networks** on top of one physical network. Instead of buying separate routers, switches, and cables for every group, you create them virtually using software.

▶ Real Example: University Campus Network

VIT Bhopal University — One Network, Four Groups

The university has one physical network infrastructure (routers, switches, cables). But different groups use it with different rules:

- **Student Network (Virtual)** — internet access, limited resources
- **Faculty Network (Virtual)** — research access, internal portals
- **Admin Network (Virtual)** — financial systems, HR databases
- **Guest Network (Virtual)** — limited internet only

All run on the **same physical hardware**, but students cannot access admin data, and guests cannot see faculty resources. This separation is created using **VLANs** (Virtual Local Area Networks).

▶ Another Example: Cloud Platforms (AWS VPC)

When a company creates a **VPC (Virtual Private Cloud)** on Amazon Web Services:

- It feels like a completely private, isolated network
- But physically it runs inside AWS's huge shared data centre
- Multiple companies share the same hardware but never see each other's data

▶ How Network Virtualization Works

Technologies used behind the scenes:

- **VLAN (Virtual LAN)** — logically separates groups on the same switch
- **SDN (Software Defined Networking)** — control network rules through software
- **Virtual Switches and Routers** — software equivalents of physical devices

Real-Life Analogy

Apartment building analogy:

- The building = physical network infrastructure
- Each flat = a separate virtual network
- All flats share plumbing and electricity, but each family has privacy

Key Points

- **Better security** — groups are isolated, can't access each other
- **Lower cost** — one physical network serves many virtual networks
- **Easy management** — configure rules through software, not hardware
- **Scalability** — add new virtual networks without buying hardware

1.6 Onsite/Outside Private and Community Cloud

Two Dimensions to Understand

Cloud deployment types are based on **where** it is located and **who** uses it:
Location: Onsite (inside your building) vs Outside (hosted elsewhere)
Access: Private (one organisation) vs Community (similar organisations share it)

▶ 1. Onsite Private Cloud

Located inside your own organisation's building. Used exclusively by that single organisation.

Example

VIT Bhopal University runs its own data centre on campus. Only university staff and systems can access it. Student exam data, internal portals, and records are stored here. No external party can reach it.

One line: "My cloud, inside my building, only for me."

▶ 2. Onsite Community Cloud

Located in one shared physical place. Used by multiple organisations with similar needs who jointly manage and fund it.

Example

Five engineering colleges in one city share a common data centre building. They use it for research databases, exam systems, and academic records. Infrastructure is at one location, costs are split, but only member colleges can access it.

One line: "Shared cloud, inside a common location, for similar groups."

▶ 3. Outside Private Cloud

Hosted externally (by a cloud provider) but used exclusively by one single organisation.

Example

A bank uses Amazon Web Services and creates a VPC (Virtual Private Cloud). The bank's data and applications run on AWS hardware physically located in AWS data centres — but only the bank can access this environment. No other company can enter.

One line: "My private cloud, but hosted by someone else outside."

▶ 4. Outside Community Cloud

Hosted by an external cloud provider. Shared by multiple similar organisations, all of whom access it over the internet.

Example

Multiple hospitals across India share a healthcare cloud hosted by Microsoft Azure. They all store patient records, medical imaging, and billing. The cloud is outside any one hospital, but only healthcare organisations in the community can access it.

One line: "Shared cloud, hosted outside, for similar organisations."

▶ Quick Comparison Table

Type	Location	Who Uses It
Onsite Private	Inside	One organisation only
Onsite Community	Inside (shared)	Multiple similar organisations
Outside Private	Outside	One organisation only
Outside Community	Outside	Multiple similar organisations

Part 2 — Platforms & Data

2.1 OpenStack in Cloud Computing

What is OpenStack?

OpenStack is an open-source cloud computing platform that allows organisations to build and manage their own **private or public cloud infrastructure** — like AWS or Azure, but on your own servers. It is free to use and modify.

▶ Main Components of OpenStack

Component	What It Does
Nova	Manages virtual machines (compute engine)
Neutron	Handles networking (IP addresses, virtual routers)
Cinder	Provides block storage (like virtual hard drives)
Swift	Object storage (files, backups, images)
Glance	Manages VM images (OS images to launch VMs from)
Keystone	Authentication and identity service (login, permissions)
Horizon	Web-based dashboard — the graphical user interface
Heat	Orchestration — automates deployment of complete environments

▶ How OpenStack Works — Step by Step

1. User requests resources via the Horizon dashboard or API
2. **Keystone** authenticates the user (checks identity and permissions)
3. **Nova** creates virtual machines on physical servers
4. **Neutron** connects the VMs to the network
5. **Cinder / Swift** provide storage for the VMs
6. **Horizon** displays everything in a clean web interface

Key Tip / Memory Trick

OpenStack = Build your own AWS. Install it on your own servers and get a full cloud management platform — all open-source and free.
Advantage: No vendor lock-in, fully customisable.
Disadvantage: Complex to install and requires skilled administrators.

2.2 Amazon Dynamo in Cloud Computing

What is Amazon Dynamo?

Amazon Dynamo (and its managed successor, **Amazon DynamoDB**) is a distributed, highly available **key-value database** designed to handle millions of requests per second without failures, even when individual servers crash.

▶ Key Concepts — Simply Explained

1. Key-Value Storage:
Instead of complex tables like SQL, Dynamo stores data as simple pairs:

- Key: `User123`
- Value: `{Name: "Mausam", Marks: 85, City: "Bhopal"}`

Retrieve any record instantly using just the key. Much faster than searching tables.

2. Distributed System:
Data is stored across many servers, not one. If one server fails, others continue serving data. No single point of failure.

3. Replication:
Every piece of data is automatically copied to multiple servers. Even if a server crashes, copies exist elsewhere.

4. Eventual Consistency:
When you update data, all copies may not update *instantly*. After a short time, all copies become consistent.

Amazon Shopping Example

During a big sale event (Diwali Sale), millions of users add items to their carts simultaneously. Dynamo ensures:

- Your cart never disappears (data replicated)
- The system doesn't crash under peak load (distributed)
- Responses are instant (key-value = no complex queries)

Feature	Dynamo	Traditional SQL DB
Structure	Key-value pairs	Tables (rows & columns)
Speed	Very fast	Moderate
Scalability	Very easy (horizontal)	Hard
Consistency	Eventual	Strong (ACID)
Complex queries	Not ideal	Excellent

2.3 Relational vs Non-Relational Databases

▶ Relational Database (SQL)

Relational Database

A **relational database** stores data in **tables** (like spreadsheets). Each table has rows (records) and columns (attributes). Tables can be linked together through relationships using foreign keys.

Structure example:

StudentID	Name	Marks	CourseID
1	Mausam	85	CS101
2	Ananya	90	CS102

Popular relational databases: MySQL, PostgreSQL, Oracle, Microsoft SQL Server

Key features: Fixed schema, uses SQL language, ACID properties (strong consistency), supports complex joins and queries.

Best for: Banking systems, ERP, college management — anything with structured data and important relationships.

▶ Non-Relational Database (NoSQL)

Non-Relational Database (NoSQL)

A **NoSQL database** stores data in flexible formats — not strict tables. Types include document (JSON), key-value, column-based, and graph databases.

Structure example (JSON document):

```
{
  "id": 1,
  "name": "Mausam",
  "posts": [
    {"text": "Hello world", "likes": 150},
    {"text": "Cloud is cool", "likes": 280}
  ]
}
```

Popular NoSQL databases: MongoDB (document), Redis (key-value), Cassandra (column), Amazon DynamoDB (key-value), Firebase Firestore (document)

Best for: Instagram, YouTube, social media — anything with large, changing, unstructured data.

▶ Comparison Table

Feature	Relational (SQL)	Non-Relational (NoSQL)
Data structure	Tables with rows/columns	JSON, key-value, graphs
Schema	Fixed (rigid)	Flexible (dynamic)
Query language	SQL	Custom APIs / query languages
Scaling	Vertical (bigger server)	Horizontal (more servers)
Consistency	Strong (ACID)	Eventual
Best use case	Banking, ERP	Social apps, big data, IoT

Real-Life Analogy

- **Relational DB** = Organised library with a catalogue system — structured, easy to find relationships
- **NoSQL DB** = Flexible storage warehouse — throw in different types of items, easy to scale out

2.4 Microsoft Azure

What is Microsoft Azure?

Microsoft Azure is Microsoft's cloud computing platform — a huge collection of online tools, servers, and services that you can rent and use over the internet without buying hardware. The same infrastructure powers Microsoft products like Teams, Office 365, and Xbox.

▶ Key Azure Services by Category

Category	Service	What It Does
Compute	Virtual Machines	Rent full computers in the cloud
	Azure App Services	Host websites and web apps
	Azure Kubernetes Service	Manage Docker containers
Storage	Blob Storage	Store files, images, videos
	Disk Storage	Storage for virtual machines
Databases	Azure SQL Database	Cloud-based SQL database
	Cosmos DB	NoSQL database, globally distributed
Networking	Virtual Network (VNet)	Private network in the cloud
	Load Balancer	Distribute traffic across servers
AI/ML	Azure AI Services	Vision, speech, language AI tools
	Machine Learning Studio	Build your own ML models
Security	Azure Active Directory	Manage users, logins, permissions

▶ Real-World Example

Building a student notes sharing platform using Azure:

- **App Service** — host the website frontend
- **Azure Functions** — backend logic (serverless)
- **Azure SQL Database** — store user accounts and metadata
- **Blob Storage** — store the actual PDF notes files
- **Azure AD** — handle login and user authentication

2.5 Google Cloud Platform (GCP) Services

What is GCP?

Google Cloud Platform is Google's cloud services collection — the same infrastructure that powers YouTube, Gmail, Google Maps, and Google Search. It is especially strong in **AI/ML** and **data analytics**.

Category	Service	Purpose
Compute	Compute Engine	Virtual machines (IaaS)
	App Engine	Upload code, Google manages rest (PaaS)
	Cloud Functions	Run code on triggers (FaaS)
	GKE	Google Kubernetes Engine — manage containers
Storage	Cloud Storage	Files, images, videos, backups
	Cloud SQL	Managed MySQL / PostgreSQL
	Firestore	NoSQL database for real-time apps
	BigQuery	Analyse massive datasets (data analytics)
Networking	VPC	Private virtual network
	Cloud CDN	Deliver content fast worldwide
AI / ML	Vertex AI	Build and train ML models
	Vision AI	Image and object recognition
	Dialogflow	Build chatbots and voice assistants

Key Tip / Memory Trick

GCP's strongest advantage = AI/ML. Google's research breakthroughs (TensorFlow, BERT, Gemini) are directly integrated into GCP services.
Pay-as-you-go pricing, same as AWS and Azure.

Part 3 — Cloud Management & Optimisation

3.1 Types of Resources in Cloud: Physical and Logical

▶ Physical Resources — The Real Hardware

Physical Resources

Physical resources are the **actual tangible hardware** inside cloud data centres. These are real machines you could physically touch if you visited the data centre building.

- **Servers (Computers):** Powerful machines doing all the computing. When your app runs "in the cloud," it runs on these physical servers.
- **Storage Devices:** Physical hard drives and SSDs where data is permanently stored.
- **Networking Equipment:** Routers, switches, fibre cables connecting everything and linking data centres to the internet.
- **Data Centres:** The entire facility housing servers, storage, networking, cooling systems, power backup generators.

Example

When you upload a file to Google Drive, it is stored on a physical hard disk in a Google data centre building — probably in a different country. You can't see it, but it exists as real hardware somewhere.

▶ Logical Resources — The Virtual Layer

Logical Resources

Logical resources are **virtual, software-based resources** created on top of physical hardware using virtualization. These are what you actually interact with as a cloud user.

- **Virtual Machines:** Software computers created by hypervisors. AWS EC2 instance = a virtual machine.
- **Containers:** Docker containers — lightweight app environments sharing host OS.
- **Virtual Networks:** Software-defined networks, VPCs, VLANs.
- **Managed Databases:** Cloud SQL, DynamoDB — appear as databases but run on many physical disks.
- **Load Balancers:** Distribute traffic across servers — software, not physical devices.
- **Cloud Storage Buckets:** Appear as simple folders but physically use many distributed disks.

Real-Life Analogy

- **Physical resources** = The kitchen (stove, fridge, utensils)
- **Logical resources** = The food delivery app interface (menu, cart, order button)

You interact with the app (logical) and never see the kitchen (physical). But the kitchen makes everything possible.

Key Tip / Memory Trick

Cloud works because logical resources are built on physical resources using **virtualization**. This makes cloud **scalable** (create VMs instantly), **flexible** (allocate exactly what you need), and **cost-effective** (no hardware purchase).

3.2 Energy Management in Cloud Computing

What is Energy Management in Cloud?

Cloud data centres have thousands of servers running 24/7 consuming enormous electricity. **Energy management** focuses on reducing power consumption while still delivering good performance.

▶ The Problem

The Idle Server Problem

Imagine a data centre with 100 servers. At 3 AM, only 10% of users are active. The other 90 servers are running at 5–10% CPU utilisation — nearly idle — but still consuming significant electricity. This is massive waste.

▶ Energy Management Techniques

- **Server Consolidation:** Move all workloads from many under-used servers onto fewer servers running at higher utilisation. Power off the now-empty servers completely.
- **Dynamic Power Management:** Reduce CPU frequency and voltage when load is low (DVFS — Dynamic Voltage and Frequency Scaling).
- **VM Migration:** Move VMs to consolidate workloads, then shut down now-empty hosts.
- **Cooling Optimisation:** Smart cooling systems that adjust to actual heat output rather than running full-blast constantly.
- **Renewable Energy:** Data centres powered by solar/wind energy (Google, Microsoft use this).

Real-Life Analogy

At night in a classroom building:
- If only 10 students are studying, you don't run AC and lights in all 20 rooms
- You move everyone to one or two rooms and close the rest
- Same idea in cloud: consolidate workloads, power off idle servers

3.3 VM Scheduling

What is VM Scheduling?

When many users request virtual machines simultaneously, the cloud must decide: **which VM runs on which physical machine, and when?** This decision process is called **VM Scheduling**.

Real-Life Analogy

You have 3 buses and 100 passengers:
- You decide who boards which bus
- Avoid overcrowding one bus while others are empty
- Same logic: distribute VMs evenly across physical servers

▶ Goals of VM Scheduling

- **Resource efficiency:** No server over-utilised or under-utilised
- **Avoid hotspots:** Don't overload one server while others are idle
- **Low latency:** Fast VM placement means faster response for users
- **Energy saving:** Pack VMs onto fewer servers to power off idle ones

▶ Types of Scheduling

Type	Description
Time-based	Decides which VM runs at which time slot
Load-based	Distributes VMs evenly based on current server load
Priority-based	High-priority tasks (paid premium) get resources first
Energy-aware	Places VMs to minimise total server energy consumption

3.4 VM Management

What is VM Management?

VM management covers the **complete lifecycle** of a virtual machine — from creation to deletion, including everything in between.

▶ VM Lifecycle Stages

1. **VM Creation:** User requests a VM. System allocates resources (CPU, RAM, disk) and creates the virtual machine.
2. **VM Allocation:** System assigns the VM to a suitable physical host server.
3. **VM Monitoring:** Continuously check CPU usage, memory consumption, network traffic, health status.
4. **VM Migration:** Move a VM from one physical server to another without shutting it down.
5. **VM Termination:** When no longer needed, the VM is shut down and its resources are returned to the pool.

▶ VM Migration — A Critical Feature

Live VM Migration

A server starts overheating due to high load. Instead of crashing, the system automatically migrates one of its VMs to a cooler, less-loaded server — **while the VM is still running**. Users notice no downtime. This is called **live migration**.

Why migrate a VM?

- Server is overloaded → move some VMs to less-loaded servers
- Server hardware is failing → evacuate VMs before it crashes
- Energy saving → consolidate VMs, power off now-empty servers
- Maintenance → move VMs off a server before maintenance window

3.5 Overhead in Cloud Computing

What is Overhead?

Overhead = the extra work, time, or resources the system must spend doing management tasks that are **not part of your actual application**. It is the "background cost" of running a cloud system.

Real-Life Analogy

You want to cook Maggi (the main task). But first you must: light the stove, boil water, wash utensils after. These are **overhead** — necessary, but not the actual cooking.
In cloud: Running your app = main task. Managing VMs, routing network, security checks = overhead.

▶ Sources of Overhead in Cloud

- **Virtualisation Overhead:** The hypervisor layer (between hardware and VMs) consumes CPU and memory just to exist and manage VMs.
- **Resource Management:** Constant monitoring, load balancing, and allocation consumes processing power.
- **Network Overhead:** Encryption/decryption, routing, transmission delays all add latency.
- **Storage Overhead:** Data replication, backup, indexing — keeping data safe takes extra storage and CPU.
- **Container/VM Startup:** Even starting a VM or container takes time and resources.

▶ How to Reduce Overhead

Technique	How It Reduces Overhead
Use containers over heavy VMs	Containers share OS kernel — far less overhead than full VMs
Efficient VM scheduling	Smart placement avoids wasted resources and management overhead
Auto-scaling	No idle machines means less management overhead
Caching	Store frequently-used data temporarily to avoid repeated computation
CDN (Content Delivery Network)	Keep content close to users to reduce network overhead
Resource right-sizing	Allocate only needed CPU/RAM — avoid over-provisioning

Key Tip / Memory Trick

You cannot eliminate overhead completely — management is necessary. But you can **minimise** it using smarter architecture, containers, and efficient scheduling.

Part 4 — Cloud Security

4.1 Security Issues in Cloud Computing

Why Security is Critical in Cloud

In cloud computing, your data, applications, and infrastructure are hosted on someone else's servers, accessible over the internet. This creates unique security challenges that don't exist in traditional on-premise systems.

▶ Major Security Issues

1. **Data Breaches:** Sensitive data exposed due to weak encryption, poor access controls, or application vulnerabilities. Example: Hackers accessing customer databases.
2. **Data Loss:** Data destroyed by ransomware, accidental deletion, or hardware failure without proper backup.
3. **Insecure IAM (Identity and Access Management):** Weak passwords, no multi-factor authentication, or overly broad permissions allow unauthorised access.
4. **Insecure APIs:** Cloud services rely on APIs. Poorly secured APIs can be exploited to take control of resources.
5. **Misconfiguration:** The most common cloud security issue. Example: Setting an S3 bucket to "public" by mistake exposes all files to the internet.
6. **Malware Injection:** Attackers inject malicious code into cloud services to steal data or disrupt operations.
7. **Insider Threats:** Employees with cloud access misuse or accidentally compromise security.
8. **Shared Technology Vulnerabilities:** Multi-tenant environments share hardware. Weak isolation between tenants can lead to data leakage.
9. **DoS/DDoS Attacks:** Flood cloud services with fake traffic to make them unavailable to real users.
10. **Compliance and Legal Issues:** Different countries have different data protection laws. Storing data in foreign clouds can create legal problems.

▶ How to Mitigate Security Risks

- Use strong encryption (data in transit and at rest)
- Enable multi-factor authentication (MFA) for all accounts
- Regularly audit and fix cloud configurations
- Apply principle of least privilege (give minimum required permissions)
- Monitor logs and alert on suspicious activity
- Keep systems, APIs, and dependencies updated
- Maintain backup and disaster recovery plans

4.2 4 Types of Security Attacks

Classic Security Attack Classification

The foundational classification of security attacks corresponds directly to the four core security properties being violated.

▶ 1. Interruption — Attack on Availability

The system or service is made **unavailable** to legitimate users.

- Like a thief blocking the entrance to a shop so no customers can enter
- **Example:** Denial of Service (DoS) attack — flooding a website with fake requests until it crashes for real users
- **Property violated:** Availability

▶ 2. Interception — Attack on Confidentiality

An attacker **secretly reads** data without permission or modification.

- Like someone secretly reading your private messages without you knowing
- **Example:** Eavesdropping on network traffic to steal passwords on an unsecured Wi-Fi network (packet sniffing)
- **Property violated:** Confidentiality

▶ 3. Modification — Attack on Integrity

An attacker **changes data** while it is being transmitted or stored.

- Like someone intercepting your letter and changing the amount written in a cheque
- **Example:** Man-in-the-Middle attack — intercepting a bank transaction and changing the recipient account number
- **Property violated:** Integrity

▶ 4. Fabrication — Attack on Authenticity

An attacker **creates fake data** or pretends to be someone they are not.

- Like someone sending an email pretending to be your professor
- **Example:** Email spoofing, creating fake login pages (phishing), injecting fake transactions
- **Property violated:** Authenticity

▶ Summary Table

Attack	What Happens	Property	Example
Interruption	Service blocked	Availability	DoS attack
Interception	Data secretly read	Confidentiality	Packet sniffing
Modification	Data changed	Integrity	Man-in-the-Middle
Fabrication	Fake data created	Authenticity	Email spoofing

Key Tip / Memory Trick

Easy memory trick: I, I, M, F
Interruption → **A**vailability **I**nterception → **C**onfidentiality

Modification → **I**ntegrity **F**abrication → **A**uthenticity

4.3 Multi-Tenancy, Threat Model, and Attack Model

▶ Multi-Tenancy

Multi-Tenant Architecture

Multi-tenancy means one cloud infrastructure serves multiple customers (tenants) simultaneously, with each customer's data and applications kept **logically isolated** from others, even though they share the same physical hardware.

Real-Life Analogy

An apartment building:
- The building = cloud infrastructure (shared)
- Each flat = a different company/user (tenant)
- All share electricity, water, structure, but live separately with locked doors

If isolation is weak, one flat could peek into another — a serious security risk.

Security risks in multi-tenancy:

- If VM isolation is weak, one tenant may access another's memory
- **Side-channel attacks:** Exploiting shared CPU cache or memory bus to leak data between tenants
- Data leakage through shared storage or networking configurations

▶ Threat Model

Threat Model

A **threat model** is a structured analysis that identifies **what can go wrong** in a system — what the assets are, who the threats are, what the vulnerabilities are, and what impact an attack would have.

Components of a threat model:

Component	Meaning	Cloud Example
Assets	What needs protection	User data, API keys, databases
Threats	Possible dangers	Hackers, malware, insider threats
Vulnerabilities	Weak points	Misconfigured storage, weak passwords
Impact	Consequence if attacked	Data breach, financial loss, downtime

▶ Attack Model

Attack Model

An **attack model** describes **how an attacker actually carries out the attack** — the specific steps, techniques, and methods used to exploit a vulnerability. If the threat model answers "what can go wrong," the attack model answers "exactly how will it happen."

Types of attacks in the attack model:

- **Passive attacks:** Only observing/listening (no data modification). Example: Traffic analysis, eavesdropping.
- **Active attacks:** Modifying, disrupting, or creating data. Example: DoS, man-in-the-middle, data injection.

Example attack model for cloud misconfiguration:

1. Attacker scans for public cloud storage buckets (network probing)
2. Finds an S3 bucket configured as "public" by mistake
3. Downloads all sensitive documents stored there
4. Uses data for financial fraud or sells it

▶ How the Three Connect

Concept	Purpose
Multi-tenancy	Creates a shared environment with inherent isolation risks
Threat model	Helps identify all possible risks in that shared environment
Attack model	Defines exactly how those risks could be exploited

4.4 Network Probing and Co-Residency

▶ Network Probing

What is Network Probing?

Network probing is when an attacker silently **scans a network** to gather information about the target — open ports, running services, IP addresses, and system versions — before launching an actual attack. It is reconnaissance, not the attack itself.

Real-Life Analogy

A thief walking around a neighbourhood at night:
- Checks which houses have lights on (active servers)
- Looks for weak locks (open ports)
- Counts windows (attack surface)
- Not stealing yet — just observing and planning

Network probing is exactly this — silent reconnaissance before the actual attack.

What attackers look for when probing:

- **Open ports:** Port 22 (SSH), Port 80 (HTTP), Port 3306 (MySQL) that are publicly accessible
- **Active IP addresses:** Which servers are online
- **Running services:** What software and what version (to find known vulnerabilities)
- **OS fingerprinting:** Which operating system the server uses

Prevention:

- Firewalls (block unauthorised port scans)
- Intrusion Detection Systems (IDS) — alert when probing is detected
- Close all unused ports
- Rate limiting on incoming connection attempts

▶ Co-Residency (Co-Location Attack)

What is Co-Residency?

Co-residency is when an attacker deliberately places their virtual machine on the **same physical server** as the victim's VM in a cloud environment. Once co-resident, the attacker can attempt side-channel attacks to extract sensitive information.

How a Co-Residency Attack Works

1. Attacker creates many VMs on AWS, targeting a specific region
2. Probes to identify which physical host a target VM is on
3. Keeps launching VMs until landing on the same physical host as the victim
4. Now co-resident — attacker's VM and victim's VM share the same CPU, memory cache, and hardware
5. Attacker uses **side-channel attacks** to observe victim's behaviour: CPU timing, cache access patterns, memory bus usage
6. Can potentially extract encryption keys, passwords, or other sensitive data through these hardware-level observations

Real-Life Analogy

You rent a flat. An attacker rents the flat right next to yours:

- They share the same walls, plumbing, electrical system
- By listening carefully to sounds through the wall (side-channel), they learn about your activities
- No direct break-in needed — proximity alone creates the vulnerability

Why co-residency is dangerous:

- No direct hacking of the victim's VM required
- Exploits **shared hardware** — CPU cache, memory bus, power channels
- Extremely difficult to detect (no intrusion into the victim's system)
- Can potentially extract cryptographic keys and session data

▶ Network Probing vs Co-Residency Together

Technique	What the Attacker Does	Goal
Network Probing	Scans network from outside to find targets	Identify vulnerable systems
Co-Residency	Gets physically close on same server	Steal data via hardware proximity

Typical combined attack sequence:

1. Network probing → identifies target server's IP, services, location
2. Co-residency attempt → attacker launches VMs to land on same physical host
3. Side-channel attack → attacker extracts sensitive data through hardware observations

Master Revision Summary — All 18 Topics

#	Topic	Core Concept	Key Example
1	Utility Computing	Pay only for what you use	Electricity bill model
2	VMs vs Containers	VM = full OS; Container = shared OS	VirtualBox vs Docker
3	SOAP in XML	XML message format for web services	Envelope + Header + Body
4	SOAP + HTTP	SOAP = message; HTTP = transport	Banking website flow
5	Network Virtualisation	One physical network, many virtual	VLANs on campus Wi-Fi
6	Cloud Deployment Types	Location (on/off) + Access (private/community)	Hospitals sharing cloud
7	OpenStack	Open-source cloud platform	Build your own AWS
8	Amazon Dynamo	Distributed key-value database	Shopping cart at scale
9	Relational vs NoSQL	SQL = tables; NoSQL = flexible	MySQL vs MongoDB
10	Microsoft Azure	Microsoft cloud platform	App Service + SQL DB
11	Google Cloud (GCP)	Google cloud, strong in AI/ML	Vertex AI + BigQuery
12	Physical vs Logical Resources	Hardware vs virtual software layer	Servers vs VMs
13	Energy Management	Reduce power in data centres	Server consolidation
14	VM Scheduling	Decide which VM runs where	Load-balanced placement
15	VM Management	Full VM lifecycle + live migration	Migration on overload
16	Overhead	Extra cost of system management	Hypervisor overhead
17	Security Issues	Breaches, misconfig, DoS, IAM	S3 bucket exposed
18	4 Attack Types	Interruption, Interception, Modification, Fabrication	DoS, sniffing, MITM, spoofing

Final Golden Rules

Utility Computing: Use it, pay for it, don't own it — like electricity.

VM vs Container: VM = whole OS inside. Container = just the app, shared OS.

SOAP + HTTP: SOAP = what the message looks like. HTTP = how it travels.

Network Virtualisation: One physical network, many virtual networks (VLANs, SDN, VPC).

Cloud Deployment: On/Off × Private/Community = 4 deployment types.

OpenStack: Open-source, build your own cloud on your own servers.

Dynamo: Key-value, distributed, eventual consistency, built for massive scale.

SQL vs NoSQL: SQL = structured, consistent, relational. NoSQL = flexible, scalable, fast.

Security Attacks: Interruption (Availability), Interception (Confidentiality), Modification (Integrity), Fabrication (Authenticity).

Multi-tenancy: Shared infrastructure, logical isolation. Weak isolation = security risk.

Network Probing: Silent reconnaissance before attack. Co-Residency: same server = side-channel risk.